Báo cáo kỹ thuật: Thiết kế máy tính RISC-V 32-bit

Nov 1, 2026 · @anhlmv

Nhóm đã thiết kế hai CPU RISC-V 32 bit trên Logisim-evolution 4.1.0, đơn chu kỳ và pipeline 5 tầng, hỗ trợ 44 lệnh (25 lệnh bắt buộc và 19 lệnh mở rộng), có I/O ánh xạ bộ nhớ và dự đoán nhánh. Cả hai CPU chạy đúng 19/19 chương trình test, khớp mô hình tham chiếu ở từng chu kỳ, và chạy được chương trình viết bằng C.

1. Giới thiệu

Dự án được phát triển qua 9 giai đoạn nối tiếp, mỗi giai đoạn thêm một nhóm chức năng và được kiểm thử xong mới sang giai đoạn sau. Báo cáo đi theo đúng trình tự đó: với mỗi chức năng hoặc nhóm lệnh, nêu những thành phần cần xây dựng để thực hiện nó.

Mục tiêu. Xây dựng một máy tính gồm bộ xử lý, bộ nhớ chương trình, bộ nhớ dữ liệu và khối điều khiển; chạy chương trình nạp từ bộ nhớ lệnh; có hai phiên bản đơn chu kỳ và pipeline 5 tầng xử lý được hazard.

Thông số. Dữ liệu, lệnh và PC 32 bit; 32 thanh ghi × 32 bit (x0 luôn bằng 0); bộ nhớ lệnh và bộ nhớ dữ liệu mỗi bộ 4 KB (1024 word); vùng I/O từ địa chỉ 0x80000000.

Công cụ. Logisim-evolution 4.1.0 để thiết kế và mô phỏng mạch, chạy test vector ở chế độ dòng lệnh. Python 3 cho bộ công cụ tự viết (assembler, mô hình tham chiếu, kiểm thử). Zig (clang + lld) để biên dịch chương trình C.

Lộ trình phát triển.

Giai đoạn	Chức năng đạt được	Thành phần xây dựng	Kiểm chứng
1. Nền tảng (mục 3)	Tính toán, lưu trữ, tạo immediate, so sánh nhánh, bộ đếm PC	`asm.py`, `rvsim.py`; ALU, RegFile, ImmGen, BranchCmp, PCUnit, IMEM, DMEM	Test vector từng module
2. Điều khiển (mục 4)	Giải mã 30 lệnh RV32I	Control (2 ROM)	197 lệnh mẫu
3. CPU đơn chu kỳ (mục 5)	Chạy trọn vẹn chương trình, CPI = 1	Circuit `CPU`, `main`; `cpu_test.py`	So từng chu kỳ với `rvsim.py`

Giai đoạn	Chức năng đạt được	Thành phần xây dựng	Kiểm chứng
4. Pipeline (mục 6)	5 lệnh chạy chồng nhau	IF_ID, ID_EX, EX_MEM, MEM_WB; `pipesim.py`	So từng chu kỳ với `pipesim.py`
5. Hazard (mục 7)	Chạy đúng không cần `nop`	ForwardUnit, WBBypass, HazardUnit, logic flush	`t9`, `t10`, `t11`
6. Byte/half-word (mục 8)	LB, LH, LBU, LHU, SB, SH	LoadExt, StoreMerge; sửa DMEM	`t13_bytes`
7. Nhân/chia (mục 9)	8 lệnh M-extension	MulDiv, IsMul; MUX sau ALU	`t14_muldiv`
8. Vào/ra (mục 10)	LED, LED 7 đoạn, UART TX, công tắc	MemMap, IO; thiết bị trong `main`	`t15_io`
9. Dự đoán nhánh (mục 11)	Nhảy đúng dự đoán không mất chu kỳ	BTB, PredRegs; sửa logic chọn PC	`t16_predict`

Mục 12 trình bày chuỗi biên dịch C; mục 13–14 là phương pháp kiểm thử và kết quả.

2. Kiến trúc tổng thể và tập lệnh

CPU hỗ trợ 44 lệnh: đủ 25 lệnh bắt buộc và 19 lệnh mở rộng. Hai phiên bản CPU dùng chung một thư viện module (`modules/riscv_modules.circ`, 20 module), nên một lỗi module chỉ cần sửa một chỗ.

Nhóm	Opcode	Lệnh	Định dạng	Giai đoạn
Số học/logic thanh ghi	0110011	ADD SUB AND OR XOR SLT SLTU SLL SRL SRA	R	1–3 (bắt buộc)
Số học/logic immediate	0010011	ADDI ANDI ORI XORI SLTI SLTIU SLLI SRLI SRAI	I	1–3 (bắt buộc)
Đọc/ghi word	0000011 / 0100011	LW / SW	I / S	1–3 (bắt buộc)
Nhảy	1100011, 1101111, 1100111	BEQ BNE JAL JALR	B / J / I	1–3 (bắt buộc)

Nhóm	Opcode	Lệnh	Định dạng	Giai đoạn
Nhảy có điều kiện mở rộng	1100011	BLT BGE BLTU BGEU	B	1–3 (mở rộng)
Nạp hằng số cao	0110111	LUI	U	2–3 (mở rộng)
Đọc/ghi byte, half-word	0000011 / 0100011	LB LH LBU LHU / SB SH	I / S	6 (mở rộng)
Nhân/chia (M-extension)	0110011, funct7 = 0000001	MUL MULH MULHSU MULHU DIV DIVU REM REMU	R	7 (mở rộng)

Định dạng lệnh (bit 31 → 0):

```
R: funct7[31:25] rs2[24:20] rs1[19:15] funct3[14:12] rd[11:7]  opcode[6:0]
I: imm[11:0]           rs1       funct3  rd        opcode
S: imm[11:5]   rs2     rs1       funct3  imm[4:0]  opcode
B: imm[12|10:5] rs2    rs1       funct3  imm[4:1|11] opcode
U: imm[31:12]                            rd        opcode
J: imm[20|10:1|11|19:12]                 rd        opcode
```

Bản đồ bộ nhớ. Kiến trúc Harvard: bộ nhớ lệnh và bộ nhớ dữ liệu tách rời.

Vùng	Địa chỉ	Thành phần	Ghi chú
Bộ nhớ lệnh	0x0000–0x0FFF	IMEM: ROM 1024 × 32	Chỉ số `PC[11:2]`
Bộ nhớ dữ liệu	0x00000000–0x7FFFFFFF	DMEM: RAM 1024 × 32, đọc không đồng bộ	Chỉ số `Addr[11:2]`, địa chỉ lớn hơn quay vòng
LED	0x80000000	IO	Ghi/đọc
LED 7 đoạn	0x80000004	IO	Ghi/đọc, 8 chữ số hex
UART TX	0x80000008	IO → TTY	Ghi = gửi ký tự; đọc = 1 (sẵn sàng)
Công tắc	0x8000000C	IO ← DIP Switch	Chỉ đọc

Luồng làm việc. Chương trình assembly (`.s`) được dịch bằng `tools/asm.py`, hoặc chương trình C được dịch bằng `tools/cc.py`, ra file `.hex` định dạng Logisim "v2.0 raw". File này được nạp vào ROM bằng *Load Image*.

[Hình 2.1: chụp cây Design của `riscv_pipe.circ`, thấy thư viện `riscv_modules` với 20 module]

3. Giai đoạn 1 – Nền tảng: công cụ và các module datapath

Giai đoạn này dựng các khối mà mọi lệnh đều cần, trước khi ghép thành CPU. Mỗi module có test vector riêng và phải đạt 100% trước khi dùng.

Công cụ phần mềm làm trước. Không có công cụ thì không nạp được chương trình và không biết kết quả đúng.

- `asm.py`: assembler có nhãn, lệnh giả (`li`, `mv`, `j`, `ret`, `beqz`…), báo lỗi kèm số dòng.
- `rvsim.py`: mô hình tập lệnh, cho trạng thái đúng ở từng lệnh; kiểm các dòng `# EXPECT` trong file `.s`.
- `run_tests.py` và các `gen_*.py`: sinh và chạy test vector qua Logisim ở chế độ dòng lệnh.

Lệnh nào cần module nào.

Module	Để thực hiện	Chân vào → ra	Cách hiện thực	Test
`ALU`	Phép tính của lệnh R, I; địa chỉ LW/SW; đích JALR; LUI	A, B (32), ALUctl (4) → Y, Zero	1 bộ cộng cho ADD/SUB (B XOR ALUctl[0], nhớ vào = ALUctl[0]); 3 Shifter; 2 Comparator (có dấu cho SLT, không dấu cho SLTU); MUX 16 đầu vào	1298 ca
`RegFile`	Mọi lệnh đọc rs1/rs2 và ghi rd	rs1, rs2, rd (5), WD, RegWrite, clk, Reset → RD1, RD2	31 Register (không có x0), Decoder 5→32 có Enable = RegWrite, 2 MUX 32→1; ghi ở cạnh lên, đọc tổ hợp	517 bước
`ImmGen`	Lệnh I, S, B, U, J	Instr, ImmSel (3) → Imm	Splitter tách/ghép bit cho từng định dạng, mở rộng dấu, MUX 8 đầu vào	268 ca
`BranchCmp`	6 lệnh branch	A, B, funct3 → Eq, Lt, Ltu, Taken	2 Comparator; MUX 8→1 theo funct3	792 ca
`PCUnit`	Nạp lệnh tuần tự, nhảy, stall	PCnext, EN, clk, Reset → PC, PC4	Register 32 bit có Enable + bộ cộng 4	129 bước
`IMEM`	Nạp lệnh	PC → Instr	ROM 1024 × 32	19 ca
`DMEM`	LW, SW	Addr, WData, MemWrite, clk → RData	RAM 1024 × 32, chỉ số `Addr[11:2]`	244 bước (bản cuối)

ALUctl (4 bit): 0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 SLT, 6 SLTU, 7 SLL, 8 SRL, 9 SRA, 10 PASS B (cho LUI).

Hai lựa chọn đáng chú ý.

- RegFile ghi ở cạnh lên, không ghi ở cạnh xuống. Với `jalr ra, 0(ra)`, nếu ghi giữa chu kỳ thì đích nhảy sẽ bị tính lại từ giá trị `ra` mới và sai.

- RAM đặt *Asynchronous read = Yes*. Mặc định của Logisim 4.1.0 là đọc đồng bộ, khiến `lw` nhận dữ liệu chậm một chu kỳ.

[Hình 3.1–3.5: chụp sơ đồ ALU, RegFile, ImmGen, BranchCmp, DMEM]

4. Giai đoạn 2 – Khối điều khiển

Khối `Control` biến 32 bit lệnh thành 10 tín hiệu điều khiển, dùng 2 ROM nối tiếp thay cho mạch logic rời, nên thêm lệnh chỉ cần sửa nội dung ROM.

- ROM chính 128 × 14 bit, địa chỉ là đủ 7 bit opcode. Bit 0 RegWrite, 1 ALUSrc, 2 MemRead, 3 MemWrite, 5:4 MemToReg, 6 Branch, 7 Jump, 8 Jalr, 10:9 ALUOp, 13:11 ImmSel.
- ROM ALU 64 × 4 bit, địa chỉ là {ALUOp, funct3, bit 30}, ra ALUctl.
- Nội dung hai ROM sinh bằng `tools/gen_roms.py`.

Lệnh	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	Jump	Jalr	ALUOp	ImmSel	Word
R-type	1	0	0	0	00	0	0	0	R	–	0x0401
I-ALU	1	1	0	0	00	0	0	0	I	I	0x0603
Load	1	1	1	0	01	0	0	0	ADD	I	0x0017
Store	0	1	0	1	–	0	0	0	ADD	S	0x080A
Branch	0	0	0	0	–	1	0	0	–	B	0x1040
JAL	1	0	0	0	10	0	1	0	–	J	0x20A1
JALR	1	1	0	0	10	0	0	1	ADD	I	0x0123
LUI	1	1	0	0	00	0	0	0	PASS B	U	0x1A03

MemToReg: 00 = kết quả ALU, 01 = dữ liệu đọc từ bộ nhớ, 10 = PC + 4.

Ba điểm thiết kế.

1. Opcode không hợp lệ, kể cả lệnh `0x00000000`, cho control word = 0. Nhờ vậy lệnh 0 là một bong bóng an toàn cho pipeline ở giai đoạn 4.
2. ROM ALU chỉ dùng bit 30 với R-type hoặc funct3 = 101, nên `addi` với immediate âm không bị hiểu nhầm thành SUB.
3. Mọi lệnh branch đều có Branch = 1; việc chọn điều kiện nằm ở `BranchCmp`, nên BLT/BGE/BLTU/BGEU không tốn thêm logic.

Một điều thuận lợi cho các giai đoạn sau: mọi lệnh load có chung opcode, nên LB/LH/LBU/LHU (giai đoạn 6) dùng lại đúng dòng "Load" của ROM; lệnh M-extension có opcode R-type (giai đoạn 7), nên Control không phải sửa.

[Hình 4.1: chụp circuit `Control` với hai ROM]

5. Giai đoạn 3 – CPU đơn chu kỳ

Ghép các module của giai đoạn 1–2 thành circuit `CPU` trong `single/riscv_single.circ`: mỗi lệnh đi hết datapath trong một chu kỳ, thanh ghi, bộ nhớ và PC cùng cập nhật ở cạnh lên. CPU có 2 chân vào (`clk` , `Reset`) và 8 chân quan sát dùng cho kiểm thử: `PC` , `Instr` , `RegWrite` , `rd` , `WB` , `MemWrite` , `MemAddr` , `MemWData` .

Để chạy từng nhóm lệnh, datapath cần những đường nào. Các nhóm được lắp và kiểm lần lượt theo thứ tự trong bảng (chương trình `test6a` – `test6d`).

Nhóm lệnh	Đường đi dữ liệu	Phần nối thêm so với nhóm trước
R-type	PC → IMEM → Splitter (rd, funct3, rs1, rs2) → RegFile → ALU → MUX WB → RegFile	PCUnit, IMEM, Splitter, Control, RegFile, ALU
I-ALU, LUI	như R-type, nhưng ngõ B của ALU lấy Imm	ImmGen, MUX ALUSrc (RD2 / Imm)
LW, SW	ALU tính địa chỉ → DMEM; SW ghi RD2; LW trả RData về MUX WB	DMEM, MUX WB 4 đầu vào (ALU, RData, PC + 4)
Branch	BranchCmp so RD1, RD2 theo funct3; bộ cộng tính PC + Imm	BranchCmp, bộ cộng đích, MUX_B chọn PC + 4 / đích khi (Branch AND Taken) OR Jump
JAL	đích = PC + Imm, ghi PC + 4 vào rd	dùng lại đường branch; MemToReg = 10
JALR	đích = (rs1 + Imm) AND 0xFFFFFFFE, ghi PC + 4	cổng AND với mặt nạ, MUX_J đặt sau MUX_B

Lựa chọn quan trọng. Branch dùng bộ so sánh riêng (`BranchCmp`) nên ALU rảnh để tính địa chỉ và đích JALR; cách này giữ nguyên được khi chuyển sang pipeline.

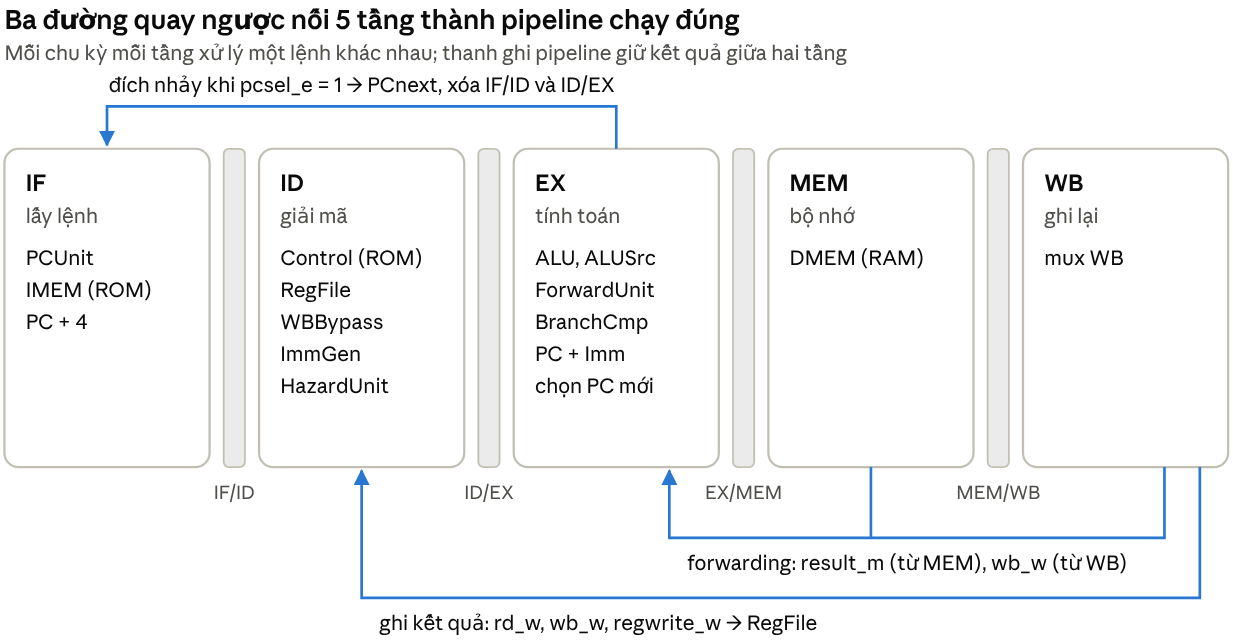
Kiểm chứng. `cpu_test.py` nạp chương trình vào bản sao file `.circ` , chạy Logisim và so 8 chân quan sát với `rvsim.py` ở mọi chu kỳ (mục 13). Cuối giai đoạn này, CPU đơn chu kỳ chạy đúng 14 chương trình RV32I.

[Hình 5.1: chụp sơ đồ circuit `CPU` đơn chu kỳ]

[Hình 5.2: chụp `main` khi đang chạy `t2_sum` , thấy Probe PC/Instr/WB và RegFile với x2 = 55]

6. Giai đoạn 4 – Pipeline 5 tầng

Datapath đơn chu kỳ được cắt thành 5 tầng IF → ID → EX → MEM → WB bằng 4 thanh ghi pipeline; mỗi chu kỳ mỗi tầng xử lý một lệnh. CPU nằm trong `pipeline/riscv_pipe.circ`, giữ cùng 8 chân quan sát và thêm `Instr_D`, `Instr_E`, `Instr_M`, `Instr_W` để thấy lệnh ở từng tầng khi demo.



Stall load-use (HazardUnit): giữ PC và IF/ID, chèn bubble vào ID/EX trong 1 chu kỳ.

Các tín hiệu mang hậu tố theo tầng (`_f`, `_d`, `_e`, `_m`, `_w`). Một khối chỉ dùng tín hiệu của tầng mình, trừ các đường quay ngược trong hình.

Để cắt được datapath, cần 4 thanh ghi pipeline. Mỗi trường `X` có chân `X_in` / `X_out`, cùng bốn chân chung `EN`, `Flush`, `clk`, `Reset`. Ở cạnh lên, thanh ghi nạp khi `EN OR Flush`; khi `Flush = 1`, các trường điều khiển và `Instr` nạp 0, tạo bong bóng.

Thanh ghi	Trường	Xóa khi Flush	Test
IF_ID	PC, PC4, Instr	Instr	146 bước
ID_EX	PC, PC4, RD1, RD2, Imm, rs1, rs2, rd, f3, Instr + 9 tín hiệu điều khiển	Instr và 9 tín hiệu điều khiển	146 bước
EX_MEM	PC, Result, WData, rd, Instr, RegWrite, MemRead, MemWrite, MemToReg	Instr và 4 tín hiệu điều khiển	146 bước

Thanh ghi	Trường	Xóa khi Flush	Test
MEM_WB	PC, Result, RData, rd, Instr, RegWrite, MemToReg	Instr, RegWrite, MemToReg	146 bước

Ba quyết định thiết kế.

1. Nhảy được quyết định ở EX. Khi không nhảy, PC tiếp tục từ `pc4_f` của tầng IF; dùng `pc4_e` sẽ nạp lại hai lệnh vừa nạp.
2. `Result` được chọn ngay ở EX (PC + 4 cho JAL/JALR, ALU cho lệnh khác), nên từ EX trở đi chỉ mang một giá trị, và forwarding từ EX/MEM đúng cả cho lệnh nhảy.
3. Các chân `EN`, `Flush` có sẵn từ đầu, ban đầu nối hằng số. Giai đoạn 5 và 9 chỉ thay hằng số bằng logic, không phải sửa module.

Kiểm chứng khi chưa xử lý hazard. `pipesim.py --lab 8` mô tả đúng cả hành vi sai khi hai lệnh phụ thuộc nhau. CPU ở giai đoạn này chạy đúng khi chèn 3 `nop` sau mỗi lệnh, và khớp mô hình từng chu kỳ cả khi không chèn.

[Hình 6.1: chụp sơ đồ circuit `CPU` pipeline; Hình 6.2: chụp `main` hiển thị lệnh ở 5 tầng]

7. Giai đoạn 5 – Xử lý hazard

Bốn cơ chế được thêm lần lượt, mỗi cơ chế bỏ được một loại `nop` phải chèn tay. Tất cả điều khiển PC và các thanh ghi pipeline qua đúng bốn tín hiệu dựng sẵn ở giai đoạn 4: `pc_en`, `ifid_en`, `ifid_flush`, `idex_flush`.

Bước	Để xử lý	Ví dụ	Module xây dựng	Giải pháp	Chi phí	Test
5a	Phụ thuộc dữ liệu cách 1–2 lệnh	`add x2,x1,x1` ngay sau lệnh ghi x1	`ForwardUnit` + 2 MUX ở EX	Lấy giá trị từ EX/MEM (`result_m`) hoặc MEM/WB (`wb_w`)	0 chu kỳ	700 ca
5b	Phụ thuộc cách 3 lệnh	lệnh đọc ở ID đúng lúc lệnh ghi ở WB	`WBBypass` sau RegFile	Lấy giá trị đang ghi thay cho giá trị cũ	0 chu kỳ	250 ca
5c	Load-use	`lw x4,0(x1)` rồi `add x5,x4,x4`	`HazardUnit` ở ID	Giữ PC và IF/ID, chèn bong bóng vào ID/EX	1 chu kỳ	400 ca
5d	Nhảy	branch có nhảy, JAL, JALR	logic `pcsel_e` ở EX	Đoán không nhảy; khi nhảy thì xóa lệnh ở IF/ID và ID/EX	2 chu kỳ	`t11`

5a – Forwarding. Điều kiện cho toán hạng 1 (toán hạng 2 thay `rs1_e` bằng `rs2_e`); EX/MEM được ưu tiên vì chứa giá trị mới hơn:

```
exA = RegWrite_m AND (rd_m != 0) AND (rd_m == rs1_e)                ->
ForwardA = 10 (lấy result_m)
wA  = RegWrite_w AND (rd_w != 0) AND (rd_w == rs1_e) AND NOT exA     ->
ForwardA = 01 (lấy wb_w)
```

Giá trị đã forward (`fa_e`, `fb_e`) được dùng ở cả năm chỗ cần giá trị thanh ghi tại EX: ngõ A của ALU, MUX ALUSrc, hai ngõ của BranchCmp và dữ liệu ghi của SW. Sau này MulDiv (giai đoạn 7) cũng lấy `fa_e`, `fb_e`.

5b – Bypass WB → ID. Sách thường cho RegFile "ghi nửa đầu, đọc nửa sau chu kỳ". Ở đây RegFile ghi ở cạnh lên (mục 3), nên `WBBypass` cho `RDx = WD` khi `RegWrite_w AND rd_w != 0 AND rd_w == rsx`. Hai cách cho cùng kết quả.

5c – Stall load-use.

```
Stall = MemRead_e AND (rd_e != 0) AND (rd_e == rs1_d OR rd_e == rs2_d)
pc_en = ifid_en = NOT Stall          idex_flush = Stall OR pcsel_e
```

Sau một chu kỳ chờ, `lw` đã tới WB và lệnh cần giá trị nhận nó qua forwarding (`ForwardA = 01`). Mạch so sánh cả `rs2` kể cả với lệnh I-type, nên có thể stall thừa nhưng không bao giờ sai.

5d – Flush khi nhảy.

```
pcsel_e = selbr_e OR jalr_e          ifid_flush = pcsel_e
```

`Stall` và `pcsel_e` không bao giờ cùng bằng 1: Stall cần lệnh ở EX là load, `pcsel_e` cần lệnh ở EX là lệnh nhảy. Thanh ghi pipeline nạp khi `EN OR Flush`, nên flush luôn có tác dụng kể cả khi đang giữ. Giai đoạn 9 thay `selbr_e` trong công thức này bằng tín hiệu "đoán sai".

[Hình 7.1: chụp ForwardUnit; Hình 7.2: HazardUnit; Hình 7.3: khu "điều khiển hazard" trong CPU pipeline]

8. Giai đoạn 6 – Lệnh byte và half-word

Để có LB, LH, LBU, LHU, SB, SH, nhóm giữ RAM đọc/ghi cả word 32 bit và thêm hai mạch tổ hợp quanh RAM. Control, ALU và các thanh ghi pipeline không phải sửa.

Thành phần	Vai trò	Cách hiện thực	Test
`LoadExt` (mới)	Đọc: lấy đúng byte/half và mở rộng dấu	`v = Word >> (A×8)` bằng Shifter; 2 Splitter lấy byte, half; 4 Bit Extender (Sign/Zero × 8/16 bit); MUX 8 đầu vào theo funct3	1248 ca
`StoreMerge` (mới)	Ghi: đọc–sửa–ghi, chỉ thay byte/half được chọn	`Merged = (Old AND NOT(K << s)) OR (D << s)`, `s = A×8`; K = 0xFF / 0xFFFF / 0xFFFFFFFF chọn theo funct3[1:0]	1088 ca
`DMEM` (sửa)	Nối hai mạch quanh RAM	Thêm chân `funct3`; `A = Addr[1:0]` bằng Bit Extender 32 → 2; RAM ra → LoadExt → RData; StoreMerge → RAM vào	244 bước
CPU đơn chu kỳ	Cấp funct3 cho DMEM	Tunnel `f3` (bit 14–12 của lệnh) vào chân `funct3`	
CPU pipeline	Cấp funct3 ở tầng MEM	Splitter lấy `f3_m` từ `instr_m` (EX_MEM đã mang sẵn `Instr`)	

```
LoadExt:    funct3 = 000 LB   001 LH   010 LW   100 LBU   101 LHU
            v = Word >> (A*8); Out = sext8(v) | sext16(v) | v | zext8(v) |
zext16(v)
StoreMerge: funct3[1:0] = 00 SB (D = WData[7:0],  K = 0xFF)
                          01 SH (D = WData[15:0], K = 0xFFFF)
                          1x SW (D = WData,       K = 0xFFFFFFFF)
```

Vì sao đọc–sửa–ghi đúng. RAM đọc không đồng bộ nên `Old` luôn là word hiện có tại địa chỉ; StoreMerge tính word mới trong cùng chu kỳ và RAM ghi lại ở cạnh lên. Không cần RAM có byte-enable.

Hai lỗi đã gặp. Logisim cấm tên chân là từ khóa VHDL (`out`, `new`), nên chân ra đặt tên `Y` và `Merged`. Khi thêm chân `funct3`, Logisim xếp lại chân của khối DMEM theo vị trí pin; pin mới phải đặt thấp nhất để các dây cũ trong CPU không bị lệch.

Kiểm chứng. `t13_bytes` đọc 4 byte có/không dấu của `0x80FF7F01`, ghi SB/SH vào từng vị trí rồi đọc lại cả word, kết thúc bằng `lbu` rồi dùng ngay kết quả (load-use): 21 giá trị EXPECT, 35 chu kỳ đơn chu kỳ và 43 chu kỳ pipeline.

[Hình 8.1: chụp LoadExt; Hình 8.2: StoreMerge; Hình 8.3: DMEM sau khi sửa]

9. Giai đoạn 7 – Nhân/chia (M-extension)

Để có 8 lệnh MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU, nhóm thêm 2 module và 1 MUX ngay sau ALU; Control, ROM và thanh ghi pipeline giữ nguyên. Các lệnh này là R-type có funct7 = 0000001, nên Control vẫn sinh RegWrite = 1, ALUSrc = 0 như lệnh R thường.

Thành phần	Vai trò	Cách hiện thực	Test
`MulDiv` (mới)	Tính 8 phép theo funct3	2 Multiplier (2′s complement cho MUL/MULH, Unsigned cho MULHU); MULHSU = MULHU − (A âm ? B : 0); Divider có dấu với upper = dấu của A; Divider không dấu 33 bit; 4 MUX xử lý chia cho 0; MUX 8 đầu vào	2760 ca
`IsMul` (mới)	Nhận biết lệnh M	`IsM = (opcode == 0110011) AND Instr[25]` ; Bit Extender lấy opcode, Shifter + Bit Extender lấy bit 25	337 ca
CPU đơn chu kỳ	Chọn kết quả	Đổi tên tunnel ra của ALU thành `aluy` ; MUX(`aluy` , `mdy` , `ism`) → `aluout`	
CPU pipeline	Chọn kết quả ở EX	MulDiv lấy `fa_e` , `fb_e` (đã forward), `f3_e` ; IsMul lấy `instr_e`	

Hai vấn đề của Logisim 4.1.0 phải xử lý.

1. Divider ở chế độ Unsigned 32 bit chia sai khi số chia ≥ 2^{31} (ví dụ `0xFFFFFFFF / 0x80000000` ra 0 thay vì 1). Nhóm mở rộng hai toán hạng lên 33 bit, dùng Divider 33 bit rồi cắt về 32 bit.
2. Chia cho 0 trong Logisim cho thương = A, dư = 0, trong khi đặc tả RISC-V yêu cầu thương = 0xFFFFFFFF, dư = A. Một Comparator `B == 0` điều khiển 4 MUX sửa lại kết quả.

Trường hợp tràn $-2^{31} / -1$ cho thương `0x80000000` , dư 0, đúng đặc tả mà không cần xử lý riêng.

Vì sao kết quả đặt ở EX. Kết quả MulDiv đi vào cùng tunnel `aluout_e` như ALU, nên forwarding, stall và ghi ngược hoạt động mà không thêm logic. Trong Logisim, bộ nhân/chia hoàn thành trong một chu kỳ; trên phần cứng thật, phép chia thường cần nhiều chu kỳ và phải stall.

Kiểm chứng và lợi ích. `t14_muldiv` kiểm các ca biên (tràn, chia cho 0, dấu của số dư), vòng giai thừa với `mul` phụ thuộc liên tiếp và ước chung lớn nhất bằng `rem`: 22 giá trị EXPECT. Cùng bài toán nhân, `t6_mul` (nhân bằng phần mềm) mất 653 chu kỳ pipeline, còn `t14b_mulhw` (lệnh `mul`) chỉ mất 26 chu kỳ.

[Hình 9.1: chụp MulDiv; Hình 9.2: MUX chọn kết quả sau ALU trong CPU pipeline]

10. Giai đoạn 8 – Vào/ra ánh xạ bộ nhớ

CPU giao tiếp với thiết bị bằng chính lệnh `lw` / `sw`: địa chỉ từ 0x80000000 trở lên đi ra thiết bị thay vì RAM (bản đồ ở mục 2). Nhờ vậy không cần thêm lệnh nào, và chương trình C truy cập thiết bị qua con trỏ `volatile`.

Thành phần	Năm ở	Vai trò	Cách hiện thực	Test
`MemMap` (mới)	Thư viện, dùng trong CPU	Giải mã vùng địa chỉ	`io = Addr ⩾ 0x80000000` (Comparator Unsigned); `RamWE = MemWrite AND NOT io`; `IOWrite = MemWrite AND io`; `RData = io ? IORData : MemRData`	106 ca
`IO` (mới)	File CPU, cạnh IMEM/DMEM	Thanh ghi thiết bị	`sel = Addr[3:2]`; Decoder 2→4; thanh ghi LED, HEX 32 bit; `TTYWE = IOWrite AND (sel = 2)`; MUX đọc [LED, HEX, 1, SW]	166 bước
CPU (sửa)	Cả hai CPU	Đưa bus ra ngoài	DMEM nhận `ramwe` thay `memwrite`; thêm pin `IORData` (vào) và `IOWrite` (ra) ở cuối danh sách chân	
Thiết bị	`main`	Người dùng thấy và bấm	LED Bar 8 đèn; 8 Hex Digit Display; TTY 8 × 32; DIP Switch 8 công tắc	

Vì sao phải chặn ghi RAM. DMEM chỉ dùng bit 2–11 của địa chỉ, nên `sw` vào 0x80000000 sẽ ghi đè ô RAM[0] nếu không có `RamWE`. `t15_io` kiểm đúng trường hợp này: ghi 0x1111 vào RAM[0], ghi I/O nhiều lần, rồi đọc lại RAM[0].

UART đơn giản. Đọc 0x80000008 luôn trả 1 (sẵn sàng); chương trình vẫn chờ trạng thái trước khi gửi, nên có thể thay bằng UART thật mà không sửa phần mềm. TTY ghi ký tự ở cạnh lên của cùng xung clock với CPU.

Thiết bị đặt ở `main`, không ở trong IO. Thiết bị năm trong mạch con chỉ thấy khi mở mạch con đó; đặt ở `main` giúp demo thấy ngay, và công tắc `SW` trở thành một pin nên test vector điều khiển được.

Kiểm chứng. `cpu_test.py` tự nhận ra CPU có pin `IORData` / `IOWrite`: nó cấp `IORData` theo mô hình (công tắc = giá trị dòng `# SW` trong file `.s`) và so `IOWrite` ở mọi chu kỳ. `t15_io` in `"Hi RISC-V!\n"` ra TTY qua vòng chờ UART, ghi LED/HEX, đọc công tắc: 10 giá trị EXPECT, gồm nội dung TTY.

[Hình 10.1: chụp circuit IO; Hình 10.2: `main` với LED, LED 7 đoạn, TTY và công tắc khi chạy demo]

11. Giai đoạn 9 – Dự đoán nhánh

Sau giai đoạn 5, mỗi lần nhảy mất 2 chu kỳ và đây là nguồn chu kỳ dôi lớn nhất (với `t4_recursion`, 171 lần flush tốn 342 chu kỳ, gấp 5 lần 66 chu kỳ stall). Giai đoạn này thêm dự đoán động: tra cứu BTB ở IF, so với kết quả thật ở EX, chỉ flush khi đoán sai. Chỉ bản pipeline được sửa.

Thành phần	Vai trò	Cách hiện thực	Test
`BTB` (mới)	16 mục: tag, đích, bộ đếm 2 bit	Dùng lại 2 khối RegFile (2 cổng đọc: IF tra cứu, EX đọc mục cũ; 1 cổng ghi). Chỉ số `16 OR PC[5:2]` (tránh ô 0); `TAGS[i] = PC OR 1` (bit 0 = hợp lệ); `TGTS[i] = {đích[31:2], bộ đếm}`; ROM 8 × 2 bit là bảng chuyển trạng thái	550 bước
`PredRegs` (mới)	Mang bit “đã đoán nhảy” theo lệnh IF → ID → EX	2 Register 1 bit, cùng quy tắc EN/Flush với IF_ID và ID_EX	192 bước

Thành phần	Vai trò	Cách hiện thực	Test
Logic chọn PC (sửa)	Theo dự đoán, sửa khi sai	Thêm MUX `seq_f`, MUX `pc1`, 2 NOT, 2 AND, 2 OR; đổi tín hiệu chọn của MUX_B và cổng OR `pcsel_e`	`t16_predict`

```
IF:  seq_f   = pred_f ? ptgt_f : pc4_f
EX:  mt_e    = selbr_e AND NOT pred_e      (đoán không nhảy, thực tế nhảy ->
về brt_e)
     mn_e    = pred_e  AND NOT selbr_e     (đoán nhảy, thực tế không nhảy ->
về pc4_e)
     pcnext  = jalr_e ? jalrt_e : (mn_e ? pc4_e : (mt_e ? brt_e : seq_f))
     pcsel_e = mt_e OR mn_e OR jalr_e      (flush 2 lệnh khi đoán sai hoặc
jalr)
Bộ đếm: nhảy +1 (tối đa 3), không nhảy −1 (tối thiểu 0); bit 1 = 1 -> đoán
nhảy
Cập nhật ở EX khi Branch OR Jump; mục mới chỉ cấp cho lệnh thực sự nhảy, bắt
đầu ở 2
```

Vì sao không cần mang đích dự đoán theo lệnh. Tag là toàn bộ PC, và đích của một lệnh branch/JAL cố định theo PC. Vậy khi đoán nhảy và thực tế nhảy thì đích đã nạp chắc chắn đúng; chỉ cần mang 1 bit `pred`.

Vì sao bit `pred` phải bị flush. Một bong bóng mang `pred = 1` sẽ bị EX hiểu là "đoán nhảy mà không nhảy" và nhảy về sai chỗ. `PredRegs` dùng đúng `ifid_en`, `ifid_flush`, `idex_flush` của thanh ghi pipeline.

JALR không được dự đoán vì đích thay đổi theo thanh ghi; `ret` vẫn mất 2 chu kỳ. Ngăn xếp địa chỉ trả về (RAS) là hướng cải tiến tiếp theo.

Kiểm chứng. `pipesim.py --lab 16` mô hình hóa đúng BTB (cả việc hai lệnh trùng chỉ số ghi đè nhau). `t16_predict` gồm vòng lặp lồng nhau, mẫu nhảy xen kẽ, gọi hàm bằng `jal`, và hai nhánh cách nhau đúng 64 byte (cùng chỉ số BTB): 463 chu kỳ, so với 555 chu kỳ khi chưa có dự đoán.

[Hình 11.1: chụp BTB; Hình 11.2: khu chọn PC mới trong CPU pipeline]

12. Chạy chương trình C

Chương trình C được biên dịch bằng clang/lld (đi kèm gói `ziglang`, cài bằng pip) thành mã máy RV32IM, rồi `tools/cc.py` chuyển thành file nạp ROM. Chương trình demo `demo/calc.c` chạy đúng từng chu kỳ trên cả hai CPU.

```
demo/calc.c --zig cc (rv32im, -O2)--> ELF --cc.py--> rom/calc.hex --Load
Image--> IMEM
              + c/start.S, c/link.ld, c/riscv_io.h
```

Ba vấn đề khi chạy mã do trình biên dịch sinh ra, và cách giải quyết.

Vấn đề	Nguyên nhân	Giải pháp
Trình biên dịch dùng AUIPC cho lời gọi hàm, CPU không hỗ trợ	Mặc định `call` = `auipc + jalr`	Bật linker relaxation (`-mcpu=generic_rv32+m+relax`) để lời gọi thành `jal`; `cc.py` kiểm mọi word và báo lỗi nếu còn lệnh ngoài tập lệnh
Chuỗi ký tự, hằng số (`.rodata`, `.data`) phải nằm trong RAM	Kiến trúc Harvard: lệnh load không đọc được ROM; Ctrl+R xóa RAM	`cc.py` sinh thêm đoạn mã ở đầu ROM ghi từng word dữ liệu vào RAM (`lui` / `addi` / `sw`), rồi nhảy tới `_start`
Không có hệ điều hành, không có `printf`	Bộ xử lý trần	`start.S` đặt `sp = 0x2000` rồi gọi `main`; `riscv_io.h` có `putc_`, `puts_`, `putu`, `to_bcd` dùng thanh ghi I/O

Bản đồ bộ nhớ cho C. `.text` trong ROM ngay sau đoạn khởi tạo; `.rodata`, `.data`, `.bss` trong RAM từ 0x1000 (DMEM chỉ dùng bit 2–11 nên 0x1000 là ô 0); ngăn xếp từ 0x2000 đi xuống.

Chương trình demo dùng đến mọi giai đoạn. `calc.c` đọc số n từ 8 công tắc, in n, n!, fib(n), tổng 1..n và tính nguyên tố ra TTY, hiển thị fib(n) dạng thập phân trên LED 7 đoạn và n trên LED. Nó dùng MUL/DIVU/REMU (giai đoạn 7) để tính và in số, SB/LBU (giai đoạn 6) cho bộ đệm ký tự, I/O (giai đoạn 8), và các vòng lặp được dự đoán nhánh (giai đoạn 9).

Công tắc n	Dòng in ra TTY	HEX	Chu kỳ đơn chu kỳ / pipeline
13	`n=13 n!=tran fib=233 tong=91 nguyen to`	00000233	840 / 983
47	`n=47 n!=tran fib=2971215073 tong=1128 nguyen to`	EEEEEEEE	– / 1303
200	`n=200 n!=tran fib=tran tong=20100`	EEEEEEEE	– / 1802

Số chu kỳ tính cho bản `-D ONESHOT` (in một lần rồi dừng), được `cpu_test.py --sw <n>` so từng chu kỳ với mô hình. Hướng dẫn từng bước năm trong tài liệu “Hướng dẫn demo toàn trình”.

[Hình 12.1: chụp `main` khi chạy `calc.hex`: TTY, LED 7 đoạn hiển thị fib(n), LED và công tắc]

13. Phương pháp kiểm thử

Kiểm thử tự động ở ba tầng, chạy bằng một lệnh `python tools/check_all.py`. Logisim được gọi ở chế độ dòng lệnh (`--test-vector`), nên sau mỗi giai đoạn, toàn bộ các phần đã làm trước đó được kiểm lại.

Tầng	Kiểm gì	Cách làm	Quy mô
Module	Từng module riêng	Test vector: vét cạn trên tập giá trị biên + ca ngẫu nhiên; module tuần tự kiểm theo từng cạnh clock (`<set>` / `<seq>`)	25 lượt (20 module + IMEM, DMEM, IO), 12.255 ca
CPU, từng chu kỳ	Mọi chu kỳ của chương trình	`cpu_test.py` so 8 tín hiệu quan sát (thêm `IOWrite` từ giai đoạn 8) với mô hình tham chiếu: `rvsim.py` (đơn chu kỳ), `pipesim.py` (pipeline)	19 chương trình × 2 CPU
Trạng thái cuối	Thanh ghi, bộ nhớ, LED/HEX/TTY khi chương trình dừng	Dòng `# EXPECT` tính tay trong mỗi file `.s`	174 giá trị

Vì sao so sánh từng chu kỳ. Một lệnh ghi sai thanh ghi rồi bị ghi đè vẫn bị phát hiện đúng chu kỳ xảy ra. Khi có sai lệch, công cụ in ra chu kỳ bị sai, lệnh đang nằm ở từng tầng IF/ID/EX/MEM/WB, và giá trị thực tế so với giá trị mong đợi.

Ý nghĩa tín hiệu quan sát ở pipeline. `PC`, `Instr` thuộc tầng IF; `RegWrite`, `rd`, `WB` thuộc tầng WB; `MemWrite`, `MemAddr`, `MemWData`, `IOWrite` thuộc tầng MEM.

Mô hình theo từng giai đoạn. `pipesim.py --lab 8/9/10/11/16` mô tả đúng hành vi của CPU ở mỗi giai đoạn, kể cả hành vi sai khi chưa xử lý hazard. Nhờ vậy CPU được kiểm ngay khi mới làm xong từng bước. Mô hình cũng được kiểm ngược bằng mạch tham chiếu dựng riêng, nhờ đó tìm ra một lỗi đọc thanh ghi sau khi ghi trong `pipesim.py`.

Bộ chương trình test.

Chương trình	Giai đoạn	Kiểm tra	Kết quả chính
`test6a` – `test6d`	3	Từng nhóm lệnh: ALU, bộ nhớ, 6 loại branch, JALR	
`t1_arith`	3	Tràn số, dịch 0/31/33, so sánh có/không dấu, immediate âm, ghi x0	26 giá trị

Chương trình	Giai đoạn	Kiểm tra	Kết quả chính
`t2_sum`	3	Vòng lặp tổng 1..10	x2 = 55
`t3_fib`	3	12 số Fibonacci ghi ra mảng	MEM[0x12C] = 89
`t4_recursion`	3	Đệ quy fib(8) qua stack	s0 = 21
`t5_sort`	3	Sắp xếp nổi bọt 8 số có dấu	mảng tăng dần
`t6_mul`	3	Nhân băng dịch và cộng	123 × 456 = 56088
`t7_hazard`	5	Chuỗi lệnh phụ thuộc sát nhau	17 giá trị
`t9_forward`	5	Forwarding, bypass, ưu tiên EX/MEM, không forward x0	13 giá trị
`t10_loaduse`	5	5 trường hợp load-use, `lw x0`, không stall thừa	7 giá trị
`t11_control`	5	Branch sau `lw`, nhảy lùi, `jal` tới `jal`, `jalr` phụ thuộc	x31 = 0
`t13_bytes`	6	LB/LH/LBU/LHU có/không dấu, SB/SH mọi vị trí, load-use với `lbu`	21 giá trị
`t14_muldiv`	7	8 lệnh M, tràn, chia cho 0, giai thừa, ước chung lớn nhất	22 giá trị
`t14b_mulhw`	7	Bài toán của `t6_mul` bằng lệnh `mul`	6 giá trị
`t15_io`	8	LED, HEX, UART TX, công tắc; ghi I/O không đè RAM	TTY = “Hi RISC-V!”
`t16_predict`	9	Vòng lồng nhau, mẫu xen kẽ, `jal`, hai nhánh trùng chỉ số BTB	7 giá trị

14. Kết quả và hiệu năng

Mọi kiểm thử đều đạt: 25/25 lượt test vector module (12.255 ca), 19/19 chương trình trên CPU đơn chu kỳ và 19/19 trên CPU pipeline có dự đoán nhánh, khớp mô hình ở từng chu kỳ. Dự đoán nhánh giảm tổng số chu kỳ thêm 14,4% so với pipeline chỉ xử lý hazard.

Kết quả module theo giai đoạn.

Giai đoạn	Module (số ca)
1	ALU (1298), RegFile (517), ImmGen (268), BranchCmp (792), PCUnit (129), IMEM (19)
2	Control (197)
4	IF_ID, ID_EX, EX_MEM, MEM_WB (146 mỗi module)
5	ForwardUnit (700), WBBypass (250), HazardUnit (400)
6	LoadExt (1248), StoreMerge (1088), DMEM (244 × 2 file)
7	MulDiv (2760), IsMul (337)
8	MemMap (106), IO (166 × 2 file)
9	BTB (550), PredRegs (192)

Số chu kỳ qua các giai đoạn của pipeline. Mỗi cột là số chu kỳ để chạy đúng chương trình; các bản chưa đủ xử lý hazard phải chèn `nop`. "Sai" nghĩa là chèn `nop` cũng không cho kết quả đúng (đích `jalr` bị lệch khi có `nop`).

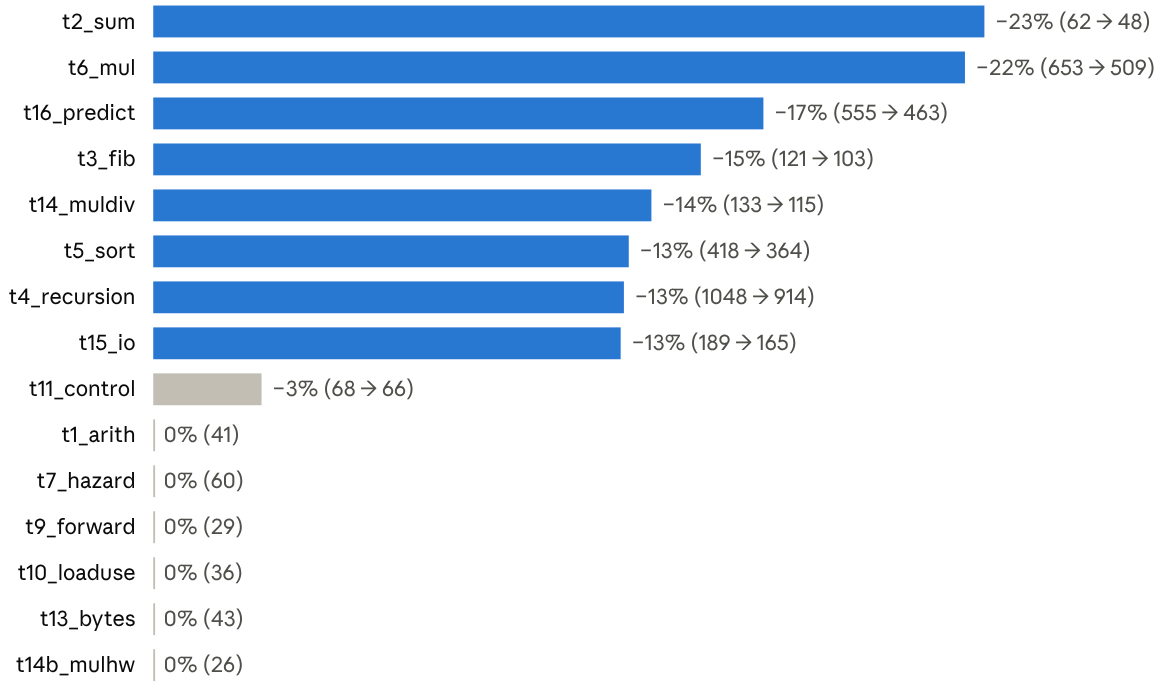
Chương trình	Số lệnh (= chu kỳ đơn chu kỳ)	GĐ 4: +3 `nop` /lệnh	GĐ 5a: `nop` sau `lw` và lệnh nhảy	GĐ 5c: `nop` sau lệnh nhảy	GĐ 5d: không `nop`
`t1_arith`	32	134	41	41	41
`t2_sum`	35	137	65	64	62
`t3_fib`	90	355	124	123	121
`t4_recursion`	637	2587	1281	1248	1048
`t5_sort`	283	1089	548	518	418
`t6_mul`	446	1703	859	859	653
`t7_hazard`	36	sai	67	66	60
`t10_loaduse`	22	94	40	36	36
`t11_control`	36	sai	sai	sai	68
`t13_bytes`	33	138	57	43	43
`t14_muldiv`	97	381	141	141	133

Chương trình	Số lệnh (= chu kỳ đơn chu kỳ)	GĐ 4: +3 `nop` /lệnh	GĐ 5a: `nop` sau `lw` và lệnh nhảy	GĐ 5c: `nop` sau lệnh nhảy	GĐ 5d: không `nop`
`t15_io`	124	486	239	235	189
`t16_predict`	356	1353	611	611	555

Giai đoạn 9 tiếp tục từ cột cuối:

Dự đoán nhánh giảm tới 23% số chu kỳ ở chương trình có vòng lặp

Số chu kỳ giảm so với pipeline chưa dự đoán (Lab 11 → Lab 16); tổng 3,482 → 2,982 chu kỳ, −14.4%



Chương trình gần như không có vòng lặp (dưới cùng) không đổi: lệnh nhảy chỉ chạy một lần nên BTB chưa kịp học.

pipesim.py, khớp CPU Logisim từng chu kỳ · 15 chương trình test

Mỗi lần đoán đúng một lệnh nhảy có nhảy tiết kiệm đúng 2 chu kỳ. Phần còn lại của số chu kỳ dôi ra là 2 chu kỳ cho mỗi `jalr` / `ret` (không dự đoán), 2 chu kỳ ở mỗi lần thoát vòng lặp, 1 chu kỳ cho mỗi load-use, và 3 chu kỳ nạp đầy pipeline.

So với CPU đơn chu kỳ. CPU đơn chu kỳ có CPI = 1 nhưng chu kỳ phải đủ dài cho cả 5 tầng. Pipeline có chu kỳ bằng tầng chậm nhất; nếu 5 tầng cân bằng thì ngắn hơn khoảng 5 lần. Với CPI 1,14–1,45 ở các chương trình lớn, thời gian chạy giảm khoảng 5 / 1,3 ≈ 3,8 lần. Đây là ước tính lý thuyết, vì Logisim không mô phỏng độ trễ cổng thật.

Bảng chi tiết do công cụ sinh: `report/ket_qua_single.md`, `report/ket_qua_pipeline.md`, `report/so_sanh.md`.

15. Kết luận

Dự án đã hoàn thành đủ yêu cầu cơ bản và bốn phần mở rộng: CPU RV32I đơn chu kỳ và pipeline 5 tầng xử lý đúng mọi hazard; thêm lệnh byte/half-word, M-extension, I/O ánh xạ bộ nhớ và dự đoán nhánh; chạy được chương trình C có nhập liệu từ công tắc và xuất ra TTY, LED 7 đoạn.

Hạn chế.

- Chưa có AUIPC, FENCE, ECALL/EBREAK, CSR và ngắt; mã C phải biên dịch với linker relaxation để tránh AUIPC.
- Bộ nhớ lệnh và dữ liệu chỉ 4 KB mỗi loại, địa chỉ vượt quá sẽ quay vòng; truy cập không thẳng hàng không báo lỗi.
- Nhân/chia hoàn thành trong một chu kỳ, chỉ hợp lý trong mô phỏng; phần cứng thật cần bộ chia nhiều chu kỳ và stall.
- UART chỉ có chiều phát; chưa có bàn phím (UART RX).
- JALR không được dự đoán, nên mỗi lần `ret` vẫn mất 2 chu kỳ.
- Logisim không mô phỏng độ trễ cổng, nên lợi ích tần số của pipeline chỉ ước tính được bằng lý thuyết.

Hướng phát triển. Ngăn xếp địa chỉ trả về (RAS) cho `ret`; quyết định nhảy sớm ở ID để giảm penalty xuống 1 chu kỳ; thêm AUIPC và ECALL để chạy mã C không cần tùy chỉnh; cache và bộ nhớ lớn hơn; UART RX và ngắt.

Bài học.

- Thiết kế sẵn các chân `EN` / `Flush` cho thanh ghi pipeline từ đầu giúp forwarding, stall, flush và dự đoán nhánh chỉ cần thêm logic, không phải sửa module.
- Đặt tên tunnel theo tầng (`_f`, `_d`, `_e`, `_m`, `_w`) và chèn mạch mới bằng cách đổi tên đúng một tunnel (`aluout` → `aluy`, `rdata` → `dmrd`) giúp mỗi phần mở rộng chỉ chạm vào vài dây.
- Kiểm thử từng chu kỳ bằng mô hình tham chiếu tìm lỗi nối dây ngay tại chu kỳ và tầng gây ra; kiểm lại toàn bộ sau mỗi giai đoạn giúp phát hiện ngay khi một thay đổi làm hỏng chức năng cũ.
- Với Logisim, thứ tự chân của khối con phụ thuộc vị trí pin; thêm pin mới ở vị trí cuối để không làm lệch dây ở mạch cha.

Phụ lục

A. Hướng dẫn chạy

Chi tiết trong `README.md`. Tóm tắt:

1. Mở `single/riscv_single.circ` hoặc `pipeline/riscv_pipe.circ` bằng Logisim-evolution 4.1.0, giữ nguyên cấu trúc thư mục.
2. Dịch chương trình: `python tools/asm.py asm/t2_sum.s -o rom/t2_sum.hex`, hoặc với C: `python tools/cc.py demo/calc.c`.
3. Nạp ROM: nhấp đúp `IMEM`, chuột phải vào ROM → *Load Image…*.
4. Chạy từ circuit `main`: Ctrl+R để reset, Ctrl+T để đi từng nửa chu kỳ, Ctrl+K để chạy tự động.
5. Kiểm thử toàn bộ: `python tools/check_all.py --jar <logisim-evolution-4.1.0-all.jar>`.

B. Kịch bản video demo (khoảng 10 phút)

Thời gian	Nội dung	Thao tác trên màn hình
0:00–0:45	Giới thiệu dự án, lộ trình 9 giai đoạn	README, cây thư mục, thư viện `riscv_modules`
0:45–2:00	CPU đơn chu kỳ	Nạp `t2_sum.hex`, Ctrl+T từng chu kỳ; mở RegFile thấy x2 tăng tới 55
2:00–3:30	Pipeline và hazard	`t9_forward`: Probe `fwda` / `fwdb`; `t10_loaduse`: `Stall` = 1, IF/ID đứng yên; `t11_control`: hai lệnh sai thành bong bóng
3:30–4:30	Lệnh byte và nhân/chia	`t13_bytes`: mở DMEM thấy SB chỉ đổi một byte; `t14_muldiv`: kết quả chia cho 0
4:30–5:30	Dự đoán nhánh	`t2_sum` trên pipeline: sau vòng đầu, lệnh nhảy không còn tạo bong bóng; so 62 → 48 chu kỳ
5:30–8:00	Demo toàn trình C	Mở `calc.c`, chạy `cc.py`, nạp `calc.hex`, Ctrl+K; gạt công tắc DIP, TTY in dòng mới, LED 7 đoạn hiện fib(n)

Thời gian	Nội dung	Thao tác trên màn hình
8:00–9:15	Kiểm thử tự động	Chạy `check_all.py`: 25 lượt module và 2 × 19 chương trình ĐẠT
9:15–10:00	Kết luận	Biểu đồ mục 14, hạn chế và hướng phát triển

C. Phân công cho nhóm 4 người

Chia theo mảng kỹ thuật, mỗi người sở hữu một mảng xuyên suốt các giai đoạn, để việc có thể làm song song và mỗi người có phần nội dung riêng trong báo cáo.

Thành viên	Mảng	Giai đoạn cơ bản	Phần mở rộng	Mục báo cáo
TV1	Datapath	ALU, RegFile, ImmGen, BranchCmp, PCUnit; lắp CPU đơn chu kỳ	MulDiv, IsMul	3, 5, 9
TV2	Điều khiển và bộ nhớ	Control (ROM, `gen_roms.py`), IMEM, DMEM	LoadExt, StoreMerge; MemMap, IO, thiết bị trong `main`	4, 8, 10
TV3	Pipeline	4 thanh ghi pipeline, lắp CPU pipeline, ForwardUnit, WBBypass, HazardUnit, flush	BTB, PredRegs	6, 7, 11
TV4	Phần mềm và kiểm thử	`asm.py`, `rvsim.py`, `pipesim.py`, `cpu_test.py`, chương trình test	Chuỗi biên dịch C, demo `calc.c`; ghép báo cáo, README, video	1, 2, 12–15

Quy tắc phối hợp.

- Thống nhất trước tên và độ rộng chân của mọi module (ví dụ `pipe_spec.py`), và bảng control word, để TV1–TV3 làm song song, TV4 viết test vector trước khi có mạch.
- File `.circ` khó gộp khi hai người cùng sửa. Mỗi người làm module trong file nháp riêng rồi chép vào `riscv_modules.circ` (Ctrl+C / Ctrl+V giữa hai cửa sổ Logisim); mỗi thời điểm chỉ một người sửa một file CPU.

- Module chỉ được ghép khi test vector đạt; sau mỗi lần ghép chạy `check_all.py` để chắc chắn không hỏng phần cũ.
- Thứ tự phụ thuộc: giai đoạn 1–2 (TV1, TV2) → CPU đơn chu kỳ → pipeline (TV3). Trong lúc chờ, TV3 làm thanh ghi pipeline và ForwardUnit/HazardUnit theo test vector; TV4 làm mô hình và chương trình test.